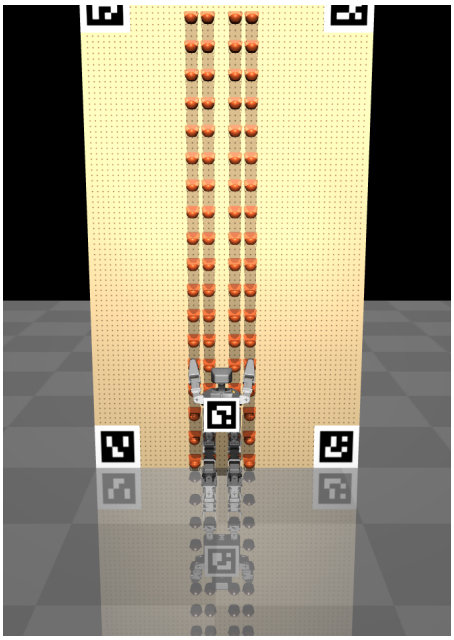
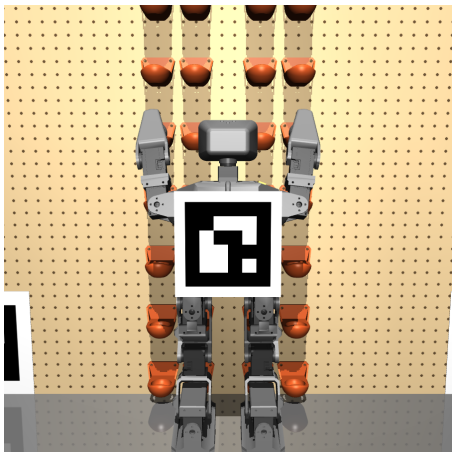
Perception par marqueurs ArUco

Hackathon Rock Climbing Robots – règlement v1.1, section 6.1

Où sont les caméras ? Où est le robot ?
Ce qui a été fait, comment ça marche, comment s'en servir





29 octobre 2026

Document de travail de l'équipe. Code dans `Hackathon/perception/`.

Table des matières

1 En bref **3**

2 Le contexte **3**

2.1 Ce que dit le règlement . . . 3
2.2 Le simulateur . . . 4
2.3 Organisation des dossiers . . . 4
2.4 Installation . . . 5

3 Les repères **5**

4 Les marqueurs ArUco **6**

4.1 Qu'est-ce qu'un marqueur ArUco ? . . . 6
4.2 La détection : `detect()` . . . 6

5 Étape 1 — Où sont les caméras ? **7**

5.1 Le modèle de la caméra . . . 7
5.2 Le problème PnP . . . 7
5.3 Le code : `camera_pose()` . . . 8

6 Étape 2 — Le marqueur ID 5 sur le robot **8**

6.1 Où le mettre et quelle taille ? . . . 8
6.2 Le repère du marqueur dans le repère du tronc . . . 9
6.3 L'ajout au simulateur : `MarkedSim` . . . 10

7 Étape 3 — Où est le robot ? **10**

7.1 La chaîne des repères . . . 10
7.2 Avec deux caméras : la triangulation . . . 10
7.3 Avec une seule caméra . . . 11
7.4 Du marqueur au tronc : `robot_pose()` . . . 12
7.5 Tout ensemble : la classe `Tracker` . . . 12

8 Les programmes et leurs commandes **13**

8.1 `demo.py` — la démonstration dans le simulateur . . . 13
8.2 `live.py` — le jour J (vraies caméras ou images) . . . 14
8.3 `doc/make_figures.py` . . . 15

9 Comment tester **15**

10 Résultats **16**

10.1 Pose des caméras (caméras par défaut, mur N1) . . . 16

10.2 Suivi du robot pendant qu'il se balance 17
10.3 Le robot en haut du mur 17
10.4 Une caméra contre deux (`live.py` sur des images du mur N3) 17

11 Pour le vrai robot **17**

12 Limites et prochaines étapes **18**
12.1 Limites connues 18
12.2 Prochaines étapes 19

A Aide-mémoire des commandes **19**

B Référence des fonctions **20**

C Petit glossaire **20**

1 En bref

Le règlement (section 6.1) propose un mode de perception *avec marqueurs ArUco* : quatre marqueurs fixés aux coins du mur donnent la position des caméras, et un cinquième marqueur, collé sur le robot, donne la position du robot. C'est ce mode qui a été programmé et testé dans le simulateur MuJoCo des organisateurs.

Ce qui marche aujourd'hui

1. **Les caméras se repèrent seules** : dans chaque image, le code trouve les marqueurs ID 1 à 4 des coins du mur et en déduit où est la caméra et où elle regarde, *sans qu'on le lui dise*. Précision : environ **2 mm** sur la position de la caméra, **0,03°** sur son orientation.
2. **Le robot porte un marqueur ID 5** sur le dos (plaque de 100 × 100 mm), ajouté au modèle du simulateur.
3. **Le robot est localisé** en croisant les deux caméras : erreur moyenne de **1,2 à 3,3 mm** sur la position du tronc, sur tout le mur (bas, milieu, haut), sur les murs N1, N2, N3, et pour des caméras placées au hasard.
4. **Le même code servira sur le vrai robot** : un script (`live.py`) lit les caméras USB, se calibre sur les marqueurs du mur et affiche la position du robot en direct.

Ce qui n'est pas encore fait. Le robot ne grimpe pas encore : la démonstration le fait seulement se balancer sur ses prises de départ pour vérifier que le suivi le suit. Il reste à détecter les prises dans les images, puis à écrire le programme de grimpe (section 12).

Les trois commandes à connaître (à lancer depuis le dossier `Hackathon/simulator`) :

```
# 1. demonstration dans le simulateur (mur N1, N2 ou N3)
uv run --with opencv-contrib-python python ../perception/demo.py N1
# 2. la meme chose avec les cameras placees au hasard (le nombre change le placement)
uv run --with opencv-contrib-python python ../perception/demo.py N2 --shuffle 3
# 3. le jour J : les deux cameras USB (gauche puis droite)
uv run --with opencv-contrib-python python ../perception/live.py 0 1
```

2 Le contexte

2.1 Ce que dit le règlement

— **Section 5.1** — deux caméras USB par mur (Intel RealSense D435i), une de chaque côté, *fournies et placées par les organisateurs*. Les équipes branchent les câbles sur leur ordinateur et récupèrent les images. **Seule l'image couleur (RGB) est autorisée** (pas la profondeur).

— **Section 5.2** — quatre marqueurs aux coins du mur, IDs 1 à 4, dictionnaire `DICT_4X4_50`. Chacun : code noir de 100 × 100 mm entouré d'une bordure blanche de 18 mm (plaque de 136 × 136 mm), le coin extérieur de la plaque sur le coin du mur. L'équipe peut ajouter un marqueur **ID 5 sur son robot**, où elle veut, de la taille qu'elle veut.

— **Section 6.1** — avec les marqueurs, les marqueurs du mur donnent la pose (position + orientation) des caméras par rapport au mur ; le robot est suivi grâce à son marqueur ID 5 (ou détecté autrement).

— **Section 8** — utiliser les marqueurs multiplie le temps par **1,25** (le mode sans marqueur, plus difficile, garde ×1). La hauteur atteinte compte avant le temps : un robot fiable avec marqueurs bat un robot sans marqueur qui tombe.

2.2 Le simulateur

Le simulateur des organisateurs (`rock-climbing-robots/simulator`, paquet Python `climbing_sim`) contient le robot Microban et ses 19 moteurs, le mur de 800 × 1400 mm avec ses prises, les quatre marqueurs des coins et deux caméras simulées qui ont les caractéristiques des D435i. Il ne contient pas de marqueur sur le robot : on l'ajoute nous-mêmes (section 6).

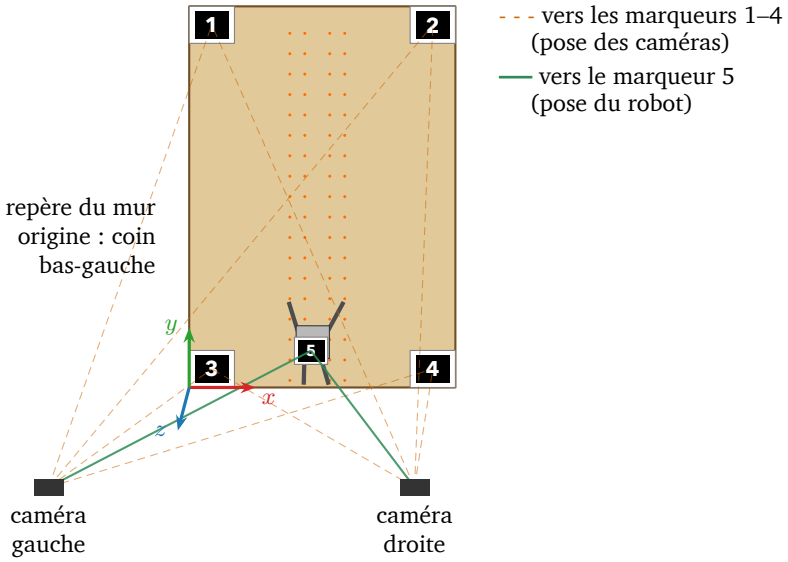


FIGURE 1 – Le dispositif (schéma, mur N1). Les deux caméras voient les quatre marqueurs des coins, ce qui leur permet de se situer ; elles voient aussi le marqueur ID 5 sur le dos du robot, ce qui permet de situer le robot.

2.3 Organisation des dossiers

Tout est dans `~/Desktop/Hackathon`. Le dépôt des organisateurs (`simulator`) n'est **pas modifié** : notre code est à part, dans `perception`.

```
Hackathon/
|-- simulator/          depot des organisateurs (climbing_sim) -- on ne le modifie pas
|-- microban/           programme du vrai robot (Rhoban, branche dev_v2)
`-- perception/         NOTRE CODE
    |-- perception.py     le traitement : detection, pose des cameras, pose du robot
    |-- robot_marker.py   le marqueur ID 5 sur le robot (simulateur) + ses dimensions
    |-- demo.py           demonstration dans le simulateur, comparee a la verite
    |-- live.py           le jour J : les vraies cameras USB (ou des images)
    |-- out/              images annotees produites par demo.py et live.py
    `-- doc/              ce document (perception_aruco.tex) et ses images
```

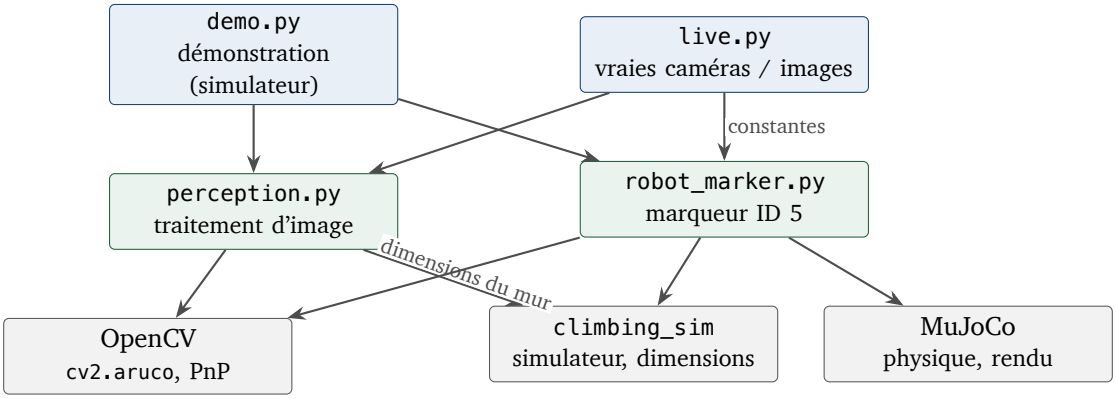


FIGURE 2 – Qui utilise quoi. `perception.py` ne dépend que d'OpenCV et des dimensions du mur : il fonctionne aussi bien sur des images simulées que sur des images réelles.

2.4 Installation

Rien de plus que ce qui était déjà installé pour le simulateur (`uv`, `git-lfs`). OpenCV n'est *pas* ajouté au projet du simulateur (pour ne pas modifier le dépôt des organisateurs) : on le demande à chaque lancement avec `--with`. `uv` le télécharge la première fois (une minute) puis le garde en cache.

```
cd ~/Desktop/Hackathon/simulator
uv run --with opencv-contrib-python python ../perception/demo.py N1
```

Le message « VIRTUAL_ENV ... does not match »

Il apparaît parce que l'environnement Python `tf_env` est activé dans ton terminal. Il est sans conséquence (`uv` utilise bien l'environnement du simulateur). Pour le faire disparaître : `deactivate`.

3 Les repères

Toute la difficulté de la localisation est de passer d'un repère à l'autre. Il y en a cinq.

Repère	Axes	Utilisé par
Mur $\mathcal{W}$	origine au coin bas-gauche de la face avant ; x à droite, y vers le haut, z sort du mur vers le grimpeur	marqueurs, caméras, résultats
Monde (MuJoCo)	même origine ; x à droite, y *entre dans* le mur, z vers le haut	simulateur, planification
Caméra $\mathcal{C}$	convention OpenCV : x à droite de l'image, y vers le bas, z devant la caméra	OpenCV
Marqueur $\mathcal{M}$	centre du code ; x à droite, y vers le haut (vu de face), z sort du marqueur	OpenCV
Tronc $\mathcal{T}$	x devant le robot (vers le mur quand il grimpe), y à sa gauche, z le long du dos	robot, IMU

Une *pose* (position + orientation) d'un repère $\mathcal{B}$ dans un repère $\mathcal{A}$ s'écrit avec une matrice de rotation R_{AB} (3×3) et une position t_{AB} : un point de coordonnées P_B dans $\mathcal{B}$ a pour coordonnées

dans $\mathcal{A}$

$$P_A = R_{AB}\,P_B + t_{AB}.$$

Les *colonnes* de R_{AB} sont les axes de $\mathcal{B}$ exprimés dans $\mathcal{A}$. Pour passer du mur au monde du simulateur, la rotation est fixe :

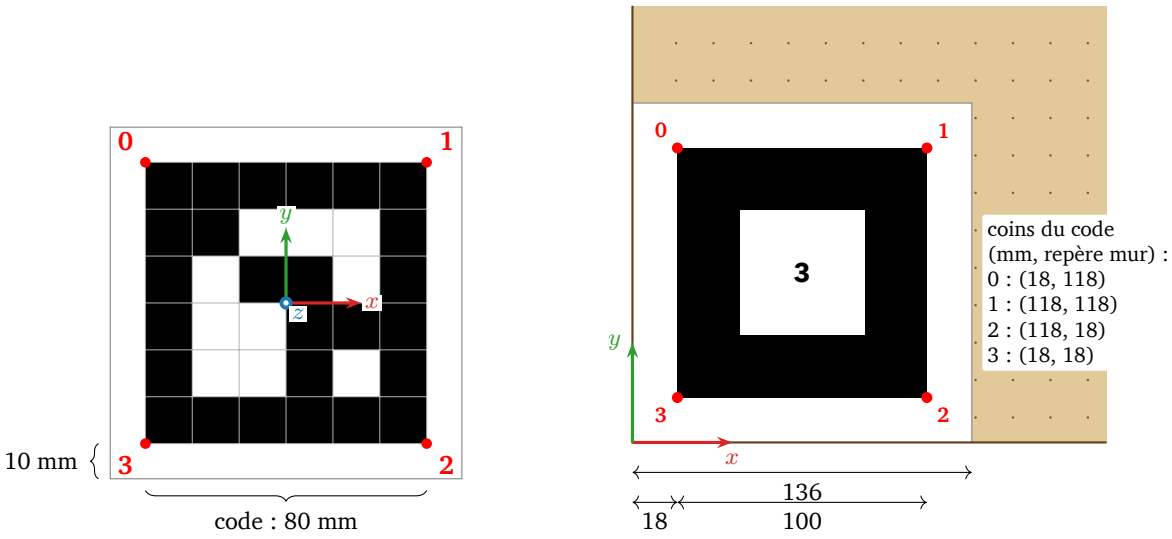
$$R_{\text{monde},\mathcal{W}} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix} \qquad (x_{\text{monde}} = x, \quad y_{\text{monde}} = -z, \quad z_{\text{monde}} = y).$$

C'est ce que fait la fonction `to_world()`.

4 Les marqueurs ArUco

4.1 Qu'est-ce qu'un marqueur ArUco ?

Un marqueur ArUco est un carré noir et blanc conçu pour être retrouvé facilement par un ordinateur. Pour le dictionnaire `DICT_4X4_50` : une grille de 6×6 cases dont la couronne extérieure est noire et dont les 4×4 cases intérieures codent un numéro (ID de 0 à 49). Autour, une bordure blanche le détache du fond. Le motif n'est pas symétrique : en le lisant, OpenCV sait *quel* marqueur il voit et *dans quel sens* il est tourné, donc il peut numéroter ses quatre coins toujours dans le même ordre.



(a) Le marqueur ID 5 du robot. En rouge, l'ordre des coins rendu par OpenCV ; au centre, le repère du marqueur (z sort de la page).

(b) Le marqueur ID 3 au coin bas-gauche du mur (motif simplifié). Ses coins ont des positions *connues* dans le repère du mur.

FIGURE 3 – Anatomie d'un marqueur. Pour chaque coin, on connaît sa position en 3D (sur le mur, ou sur le marqueur) et OpenCV donne sa position en pixels dans l'image : c'est cette double information qui permet de tout localiser.

4.2 La détection : `detect()`

OpenCV fait tout le travail : l'image est passée en niveaux de gris, les quadrilatères sombres sont repérés, leur contenu est lu et comparé au dictionnaire. On active l'*affinage sous-pixel* des coins (`CORNER_REFINE_SUBPIX`) : la position des coins est alors précise à une fraction de pixel, ce qui compte beaucoup pour la précision finale.

```
_params = cv2.aruco.DetectorParameters()
_params.cornerRefinementMethod = cv2.aruco.CORNER_REFINE_SUBPIX
_detector = cv2.aruco.ArucoDetector(cv2.aruco.getPredefinedDictionary(cv2.aruco.
    DICT_4X4_50), _params)

def detect(image_rgb) -> dict[int, np.ndarray]:
    gray = cv2.cvtColor(np.ascontiguousarray(image_rgb), cv2.COLOR_RGB2GRAY)
    corners, ids, _ = _detector.detectMarkers(gray)
    if ids is None:
        return {}
    return {int(i): c.reshape(4, 2) for i, c in zip(ids.ravel(), corners)}
```

Le résultat est un dictionnaire `{ID: coins}`, par exemple `{1: ..., 2: ..., 3: ..., 4: ..., 5: ...}`, chaque entrée contenant les 4 coins (4×2 pixels) dans l'ordre de la figure 3.

Les positions 3D des coins des marqueurs du mur sont calculées par `wall_marker_corners(id)` à partir des dimensions du règlement (fonction `aruco.corner_plate_origin` du simulateur). La face avant des plaques est à $z = 1$ mm du mur dans le simulateur (plaques de 1 mm d'épaisseur) : constante `PLATE_Z`.

5 Étape 1 – Où sont les caméras ?

5.1 Le modèle de la caméra

Une caméra (modèle « sténopé », celui d'OpenCV) transforme un point 3D en pixel en deux temps :

1. **les extrinsèques** (R, t) — où est la caméra et où elle regarde — font passer le point du repère du mur au repère de la caméra : $P_C = R\,P_W + t$;
2. **les intrinsèques** K — propres à la caméra (focale f_x, f_y et centre optique c_x, c_y en pixels) — le projettent sur l'image :

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \propto K\,P_C, \qquad K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}.$$

Les intrinsèques se mesurent une fois pour toutes (calibration de la caméra, ou valeurs du fabricant). Les extrinsèques changent dès qu'on bouge la caméra : ce sont elles qu'on cherche.

5.2 Le problème PnP

On connaît n points 3D P_i (les coins des marqueurs du mur) et leurs pixels p_i dans l'image (donnés par `detect()`). Trouver la pose (R, t) de la caméra qui explique ces pixels s'appelle le problème **PnP** (*Perspective-n-Point*) :

$$(R, t) = \arg\min_{R,t} \sum_{i=1}^{n} \left\| \pi\big(K(RP_i + t)\big) - p_i \right\|^2$$

où π divise par la troisième coordonnée. Avec les 4 marqueurs, on a $n = 16$ coins, répartis aux quatre coins du mur : c'est très contraint, donc très précis.

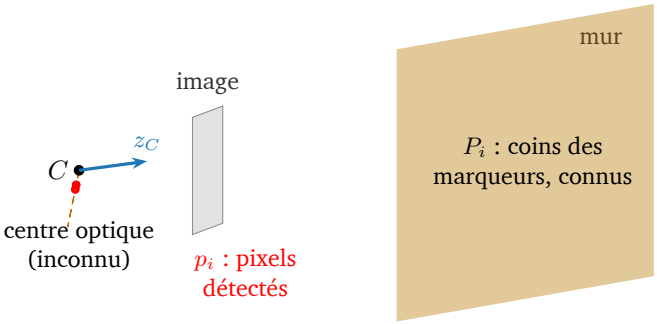


FIGURE 4 - Le problème PnP : on connaît les points du mur (P_i) et l'endroit où ils tombent dans l'image (p_i) ; on cherche la position et l'orientation de la caméra qui alignent chaque rayon.

5.3 Le code : `camera_pose()`

```python
def camera_pose(markers: dict, K, dist=None):
    ids = [i for i in WALL_IDS if i in markers]          # les marqueurs 1–4 vus
    if not ids:
        return None
    obj = np.concatenate([wall_marker_corners(i) for i in ids])  # points 3D (repere mur)
    img = np.concatenate([markers[i] for i in ids]).astype(np.float64)  # pixels
    ...
    _, rvec, tvec = cv2.solvePnP(obj, img, K, dist, flags=cv2.SOLVEPNP_IPPE)
    rvec, tvec = cv2.solvePnPRefineLM(obj, img, K, dist, rvec, tvec)
    proj, _ = cv2.projectPoints(obj, rvec, tvec, K, dist)
    rms = float(np.sqrt(np.mean(np.sum((proj.reshape(-1, 2) - img) ** 2, axis=1))))
    return rvec.ravel(), tvec.ravel(), rms
```

— `SOLVEPNP_IPPE` est une méthode exacte pour des points tous dans un même plan (ici le mur) ; elle donne une première solution.

— `solvePnPRefineLM` l'affine par moindres carrés (Levenberg–Marquardt).

— `rvec` est la rotation R sous forme compacte (axe × angle, convertie par `cv2.Rodrigues`).

— `rms` est l'**erreur de reprojection** : on reprojette les 16 coins avec la pose trouvée et on mesure l'écart moyen avec les pixels détectés. Autour de 0,4 pixel : la pose explique très bien l'image. Plusieurs pixels : problème (mauvaises intrinsèques, marqueur mal détecté…).

— La position de la caméra dans le repère du mur s'en déduit : $C = -R^\top t$.

À retenir

Le code n'a besoin de connaître *que* les intrinsèques des caméras. Leur position est retrouvée à chaque essai à partir des marqueurs du mur : les organisateurs peuvent placer les caméras où ils veulent, tant que chacune voit les quatre coins.

6 Étape 2 – Le marqueur ID 5 sur le robot

6.1 Où le mettre et quelle taille ?

Quand le robot grimpe, il fait face au mur : les caméras voient donc son **dos**. Le marqueur va sur le dos du tronc, la partie la plus stable du robot (les bras et les jambes bougent sans arrêt).

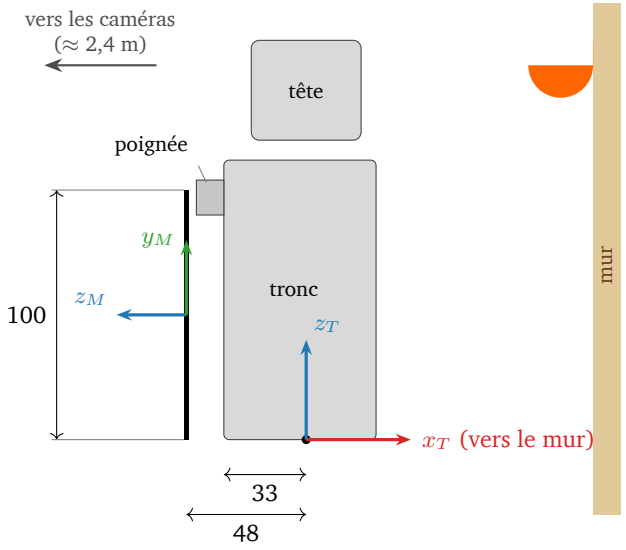
Le dos du tronc a été mesuré dans le modèle du simulateur en lançant des rayons vers lui (fonction `mj_ray` de MuJoCo) : il est **plat à 33 mm derrière l'origine du tronc**, sur environ 70 mm de large et de $z = 0$ à $z = 100$ mm ; une poignée dépasse jusqu'à 44 mm. La plaque est donc placée **à 48 mm**, juste derrière la poignée.

— plaque de **100 × 100 mm** : code noir de **80 mm** + bordure blanche de 10 mm ;

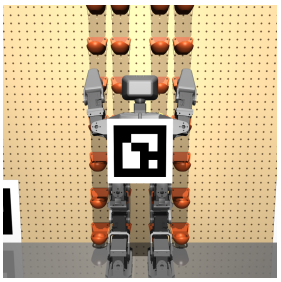
— centre du code à 50 mm au-dessus de l'origine du tronc, au milieu du dos ;

— dans le simulateur, la plaque est purement visuelle : pas de collision, pas de masse.

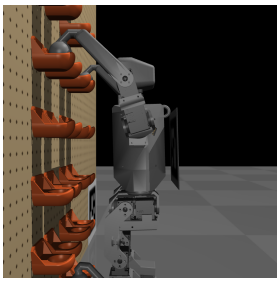
Plus le marqueur est grand, plus il couvre de pixels et plus la mesure est précise. Vu à 2,4 m par une D435i en 1280 × 720, le code de 80 mm fait environ 30 pixels de côté (5 pixels par case) : c'est suffisant, et c'est à peu près le maximum que permet la largeur du dos.



(a) Vue de côté (mm, proportions approximatives) : la plaque à 48 mm derrière l'origine du tronc.



(b) Vu de dos.



(c) Vu de côté.

FIGURE 5 – Le marqueur ID 5 sur le dos du robot, dans le simulateur.

6.2 Le repère du marqueur dans le repère du tronc

Pour passer de la pose du marqueur à la pose du robot, il faut savoir exactement comment le marqueur est fixé sur le tronc. Vu de derrière le robot (comme le voient les caméras) :

— l'axe x_M du marqueur (vers la droite de l'image) est la *droite* du robot, soit $-y_T$;

— l'axe y_M (vers le haut du marqueur) est le haut du robot, soit $+z_T$;

— l'axe z_M (qui sort du marqueur vers la caméra) va vers l'arrière du robot, soit $-x_T$.

Ces trois vecteurs sont les colonnes de la matrice `R_TRUNK_MARKER`, et la position du centre du code est `P_TRUNK_MARKER` :

$$R_{TM} = \begin{pmatrix} 0 & 0 & -1 \\ -1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \qquad t_{TM} = \begin{pmatrix} -48 \\ 0 \\ 50 \end{pmatrix} \text{ mm}.$$

Sur le vrai robot

Ces deux valeurs doivent être **mesurées sur le vrai robot** une fois le marqueur collé, et reportées dans `robot_marker.py`. Une erreur de 5 mm sur `P_TRUNK_MARKER` donne directement 5 mm d'erreur sur la position du robot, quelle que soit la qualité des caméras. Idem pour la taille du code (`CODE = 0.080`) : mesure-la sur l'impression.

6.3 L'ajout au simulateur : `MarkedSim`

Le simulateur construit sa scène MuJoCo avec `build_spec()`. La fonction `add_robot_marker(spec)` ajoute au corps `trunk` une boîte très fine (1 mm) dont la face extérieure porte l'image du marqueur (texture générée par OpenCV avec `cv2.aruco.generateImageMarker`). La boîte est orientée par R_{TM}, pour que l'image apparaisse à l'endroit vue de derrière.

```
class MarkedSim(ClimbingSim):
    """ClimbingSim with the robot's marker ID 5 on its back (same arguments)."""
    def __init__(self, *args, **kwargs):
        _sim_module.build_spec = lambda **kw: add_robot_marker(build_spec(**kw))
        try:
            super().__init__(*args, **kwargs)
        finally:
            _sim_module.build_spec = build_spec
```

`MarkedSim` s'utilise exactement comme `ClimbingSim` (`MarkedSim("N1")`, `reset_hanging()`, `step()`, `render()`…). Pendant sa construction, il remplace temporairement la fonction qui fabrique la scène par une version qui ajoute le marqueur, puis remet l'originale. Le dépôt du simulateur n'est pas modifié.

La fonction `marker_image()` rend l'image du marqueur avec sa bordure : c'est elle qu'il faudra imprimer pour le vrai robot, à 100 × 100 mm.

7 Étape 3 – Où est le robot ?

7.1 La chaîne des repères

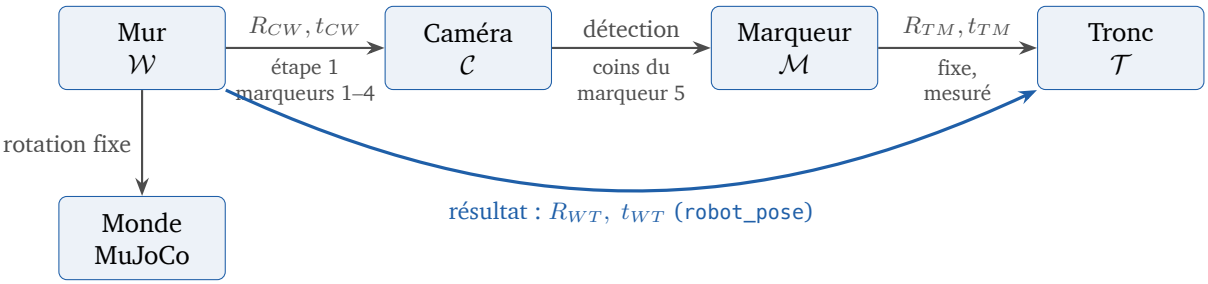


FIGURE 6 – Pour connaître la pose du tronc dans le repère du mur, on enchaîne : mur → caméra (étape 1), caméra → marqueur (détection), marqueur → tronc (fixation mesurée).

7.2 Avec deux caméras : la triangulation

Chaque caméra calibrée transforme un pixel en un *rayon* dans l'espace : le coin du marqueur est quelque part sur ce rayon, mais une seule caméra ne dit pas à quelle distance. Avec deux caméras, le

coin est à l'**intersection des deux rayons**. C'est la triangulation (`cv2.triangulatePoints`, méthode linéaire DLT), appliquée à chacun des 4 coins du marqueur ID 5.

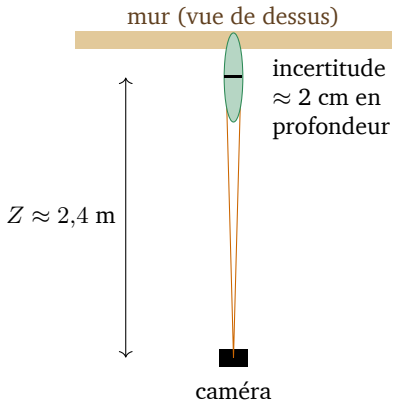
On obtient 4 points 3D dans le repère du mur. On connaît aussi la forme du marqueur (un carré de 80 mm, `square_corners(0.08)`). Il reste à trouver la rotation et la translation qui superposent au mieux le carré théorique sur les 4 points mesurés : c'est l'**algorithme de Kabsch** (`_rigid_fit`), qui résout ce problème exactement avec une décomposition en valeurs singulières :

$$H = \sum_i (a_i - \bar{a})(b_i - \bar{b})^\top = U\Sigma V^\top, \qquad R = V\,\mathrm{diag}\big(1, 1, \det(VU^\top)\big)\,U^\top, \qquad t = \bar{b} - R\,\bar{a},$$

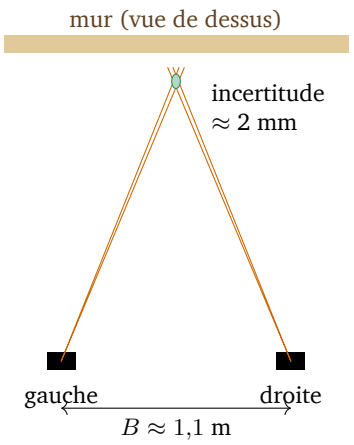
où a_i sont les coins du carré théorique et b_i les coins triangulés. Le det garantit une vraie rotation (pas un miroir).

7.3 Avec une seule caméra

Si une seule caméra voit le marqueur (l'autre est masquée par un bras, par exemple), `marker_pose` résout un petit PnP sur les 4 coins du marqueur seul (`SOLVEPNP_IPPE_SQUARE`), puis le ramène dans le repère du mur avec la pose de la caméra. Ça marche, mais c'est **beaucoup moins précis en profondeur**.



(a) Une caméra : la distance vient de la *taille apparente* du marqueur (environ 30 pixels), mesure peu précise.



(b) Deux caméras : la position est à l'intersection des rayons ; l'écart B entre les caméras rend la mesure de profondeur précise.

FIGURE 7 – Pourquoi deux caméras valent beaucoup mieux qu'une.

Un calcul d'ordre de grandeur. Avec une focale $f \approx 924$ pixels, un marqueur de $L = 80$ mm à $Z \approx 2{,}4$ m fait $s = fL/Z \approx 30$ pixels. Une erreur δs sur sa taille apparente donne une erreur de profondeur

$$\delta Z_{1\text{ caméra}} \approx \frac{Z^2}{f\,L}\,\delta s \approx \frac{2{,}4^2}{924 \times 0{,}08} \times 0{,}25 \approx 2\text{ cm} \qquad (\text{pour } \delta s \approx 0{,}25 \text{ pixel}).$$

Avec deux caméras écartées de $B \approx 1{,}1$ m, c'est l'écartement B qui remplace la taille du marqueur :

$$\delta Z_{2\text{ caméras}} \approx \frac{Z^2}{f\,B}\,\delta d \approx \frac{2{,}4^2}{924 \times 1{,}1} \times 0{,}25 \approx 1{,}5\text{ mm}.$$

C'est exactement ce que montrent les tests (section 10) : environ 2 cm avec une caméra, environ 2 mm avec deux. Et c'est pourquoi les caméras les plus rapprochées donnent les moins bons résultats.

7.4 Du marqueur au tronc : `robot_pose()`

La triangulation donne la pose du marqueur dans le repère du mur, (R_{WM}, t_{WM}). Le marqueur étant fixé sur le tronc par (R_{TM}, t_{TM}), on inverse cette fixation :

$$R_{WT} = R_{WM}\, R_{TM}^{\top}, \qquad t_{WT} = t_{WM} - R_{WT}\, t_{TM}.$$

```python
def robot_pose(views: list, size: float, R_trunk_marker, p_trunk_marker):
    pose = marker_pose(views, size)        # le marqueur dans le repere du mur
    if pose is None:
        return None
    R_wm, t_wm = pose
    R = R_wm @ np.asarray(R_trunk_marker, float).T
    return R, t_wm - R @ np.asarray(p_trunk_marker, float)
```

Enfin `to_world(R, p)` convertit cette pose dans le repère du simulateur, pour la comparer à la vérité (`sim.trunk_pose()`) ou pour la donner au programme de grimpe.

7.5 Tout ensemble : la classe `Tracker`

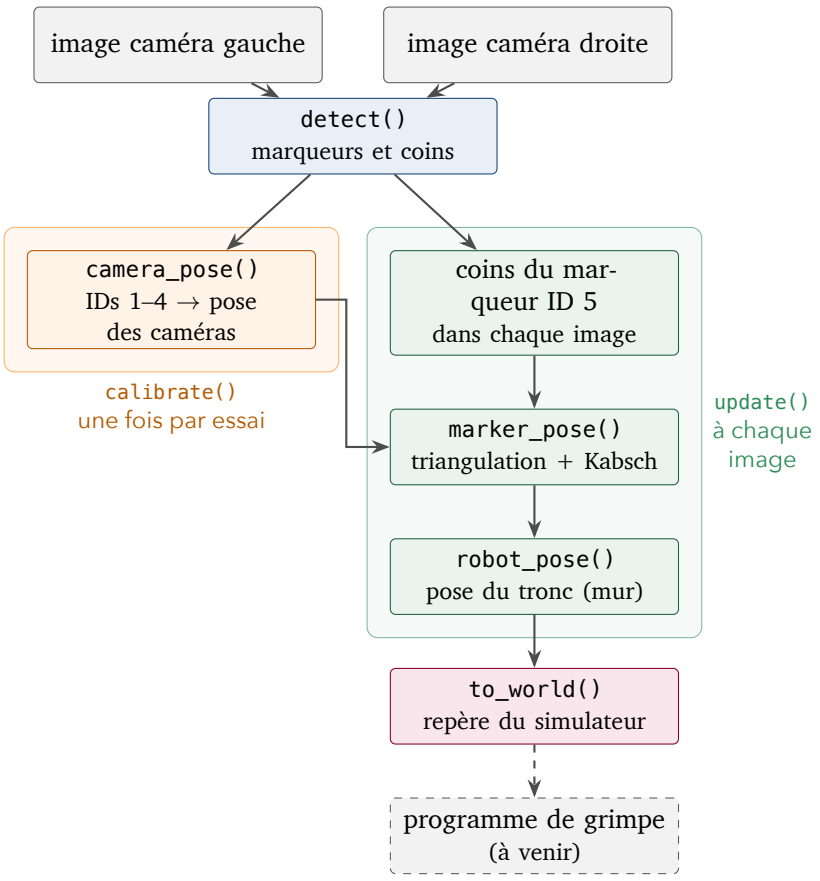


Figure 8 – Le traitement complet. La calibration des caméras (orange) se fait une fois au début de chaque essai ; le suivi du robot (vert) à chaque nouvelle image, en utilisant la pose des caméras trouvée par la calibration.

```python
from perception import Tracker, to_world
from robot_marker import CODE, R_TRUNK_MARKER, P_TRUNK_MARKER
```

```
tracker = Tracker({"cam_left": K_gauche, "cam_right": K_droite},   # intrinseques
                  CODE, R_TRUNK_MARKER, P_TRUNK_MARKER)
tracker.calibrate({"cam_left": img_g, "cam_right": img_d})   # debut d'essai
pose = tracker.update({"cam_left": img_g, "cam_right": img_d})  # a chaque image
if pose is not None:
    R, p = pose                    # tronc dans le repere du mur (p en metres)
    R_monde, p_monde = to_world(R, p)
```

8 Les programmes et leurs commandes

Toutes les commandes se lancent depuis `~/Desktop/Hackathon/simulator` (pour que `uv` utilise l'environnement du simulateur), avec `--with opencv-contrib-python` pour ajouter OpenCV.

8.1 `demo.py` – la démonstration dans le simulateur

```
uv run --with opencv-contrib-python python ../perception/demo.py [N1|N2|N3] [--shuffle
    SEED]
```

Option	Effet
`N1`, `N2`, `N3`	le mur (niveaux de difficulté 1 à 3 du règlement) ; `N1` par défaut
`--shuffle SEED`	place les deux caméras au hasard devant le mur : entre 1,9 et 3 m du mur, entre 0,5 et 1,4 m de haut, une de chaque côté, visée près du centre ; un placement est retiré tant qu'une caméra ne voit pas les quatre coins. `SEED` (un nombre entier) choisit le tirage : le même nombre redonne le même placement.

Ce que fait le script.

1. Il crée la simulation avec le marqueur (`MarkedSim`) et accroche le robot à ses quatre prises de départ (`reset_hanging`).
2. **Étape 1** : il prend une image de chaque caméra, calibre les caméras (`tracker.calibrate`) et compare la pose trouvée à la vraie pose des caméras du simulateur.
3. **Étape 2** : pendant 3 secondes simulées, il fait se balancer le robot (le tronc oscille de ±30 mm de côté et ±15 mm en hauteur, les quatre mains et pieds restant dans leurs prises). Toutes les 0,3 s il prend les deux images, calcule la pose du tronc (`tracker.update`) et la compare à la vraie.
4. Il enregistre dans `perception/out/` les images annotées (`cam_left.png`, `cam_right.png`) et les images brutes, telles que les caméras les voient (`cam_left_raw.png`, `cam_right_raw.png`).

Lire la sortie. Voici ce qu'affiche la démo sur le mur N1 avec les caméras par défaut :

```
cam_left at x -0.15 m, 0.90 m high, 2.40 m from the wall
cam_right at x +0.95 m, 0.90 m high, 2.40 m from the wall
wall N1: camera poses from the wall markers (ground truth: the simulator's cameras)
  cam_left: markers [1, 2, 3, 4, 5], reprojection 0.38 px, position error 1.6 mm, angle error 0.033 deg
  cam_right: markers [1, 2, 3, 4, 5], reprojection 0.38 px, position error 2.3 mm, angle error 0.008 deg
trunk pose from marker ID 5 (ground truth: the simulated trunk)
  t= 1.02 s: trunk at [ 388 -117  171] mm (true [ 385 -115  172]), error   3.2 mm 3.44 deg, seen by [...]
  ...
mean error 1.6 mm (max 3.2), 1.53 deg (max 3.44) over 11 frames
annotated images: /Users/.../Hackathon/perception/out
```

Ligne	Signification
`cam_left at x ...`	où le simulateur a placé la caméra (la vérité, que le code ne connaît pas)
`markers [1, 2, 3, 4, 5]`	les marqueurs détectés dans l'image : il faut 1–4 (coins) et 5 (robot)
`reprojection 0.38 px`	écart moyen entre les coins détectés et les coins reprojetés : < 1 pixel = bon
`position error 1.6 mm`	distance entre la position calculée de la caméra et la vraie
`angle error 0.033 deg`	écart d'orientation de la caméra
`trunk at [x y z] mm`	position du tronc calculée par les caméras, dans le repère monde
`(true [x y z])`	la vraie position dans le simulateur
`error 3.2 mm 3.44 deg`	écart de position et d'orientation entre les deux
`seen by [...]`	les caméras qui voient le marqueur ID 5 (idéalement les deux)
`mean error ...`	le bilan sur toutes les images

8.2 `live.py` – le jour J (vraies caméras ou images)

```
# deux cameras USB, gauche puis droite
uv run --with opencv-contrib-python python ../perception/live.py 0 1
# avec la calibration des cameras
uv run --with opencv-contrib-python python ../perception/live.py 0 1 --cameras cams.json
# sur deux images (photos, ou images du simulateur), sans fenetre
uv run --with opencv-contrib-python python ../perception/live.py gauche.png droite.png --
    once
```

Argument	Effet
`0 1` (sources)	une ou deux sources, **la caméra gauche en premier** : un numéro de caméra USB ou un fichier image. Une RealSense apparaît comme plusieurs caméras (couleur, infrarouge…) : essayer 0, 1, 2… jusqu'à voir l'image couleur.
`--cameras cams.json`	la calibration de chaque caméra (format de l'éditeur de caméras du simulateur : `fx`, `fy`, `cx`, `cy`, et facultativement `dist`, les coefficients de distorsion d'OpenCV). Sans ce fichier : les valeurs théoriques de la D435i (moins précis).
`--once`	une seule image par caméra, pas de fenêtre : affiche le résultat et enregistre `out/live_cam_left.png` et `out/live_cam_right.png`

Les touches dans la fenêtre. **c** : recalibrer les caméras (à faire au début de chaque essai, robot en place) ; **s** : sauvegarder les images annotées ; **q** : quitter.

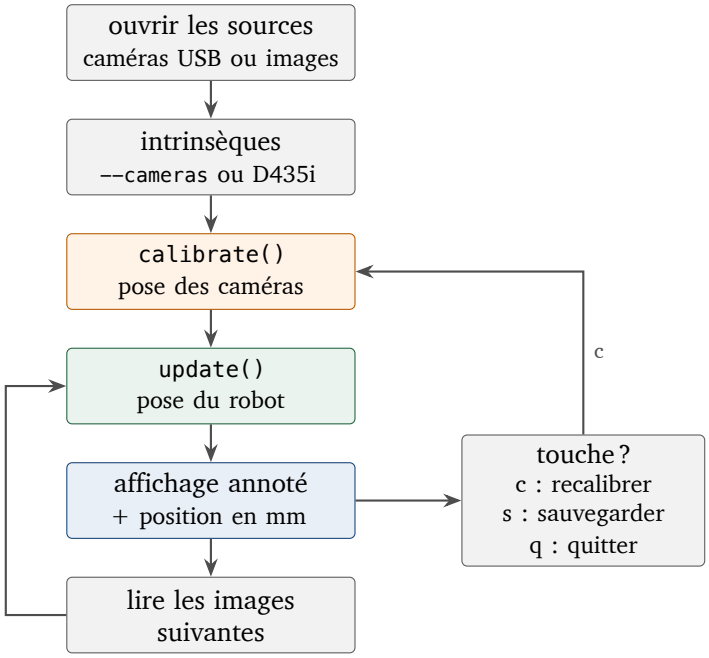


FIGURE 9 – La boucle de `live.py`.

La ligne d'état indique par exemple :

```
robot: x    385 mm, height    172 mm,   116 mm from the wall, seen by ['cam_left', 'cam_right']
```

Les positions sont dans le **repère du mur** : x depuis le bord gauche, hauteur depuis le bas, distance au mur (devant).

8.3 `doc/make_figures.py`

Régénère les images de ce document à partir du simulateur (`perception/doc/img/`). Le document se compile avec `latexmk -xelatex perception_aruco.tex` dans `perception/doc/`.

9 Comment tester

1. **La démonstration de base.**

   ```
   cd ~/Desktop/Hackathon/simulator
   uv run --with opencv-contrib-python python ../perception/demo.py N1
   ```

   C'est réussi si : chaque caméra voit `[1, 2, 3, 4, 5]`, la reprojection est sous 1 pixel, et `trunk at` est à quelques millimètres de `true`. C'est raté si on voit `marker not seen` ou des erreurs de plusieurs centimètres.

2. **Les images annotées.**

   ```
   open ../perception/out/cam_left.png ../perception/out/cam_right.png
   ```

   Chaque marqueur détecté est encadré en vert avec son numéro ; sur le dos du robot, trois flèches dessinent le repère du tronc calculé (rouge vers le mur, vert vers la gauche du robot,

bleu vers le haut). Elles doivent être collées au dos du robot.

3. **Les autres murs**, puis **d'autres placements de caméras** :

```
uv run --with opencv-contrib-python python ../perception/demo.py N3
uv run --with opencv-contrib-python python ../perception/demo.py N2 --shuffle 1
uv run --with opencv-contrib-python python ../perception/demo.py N2 --shuffle 2
```

4. **Le script du jour J sur des images** (sans caméra) : la démo a laissé dans `out/` les images brutes des deux caméras.

```
uv run --with opencv-contrib-python python ../perception/live.py \
    ../perception/out/cam_left_raw.png ../perception/out/cam_right_raw.png --once
```

Le résultat (`robot: x ... mm, height ... mm, ... mm from the wall`) doit correspondre à la position du robot au milieu du balancement de la démo.

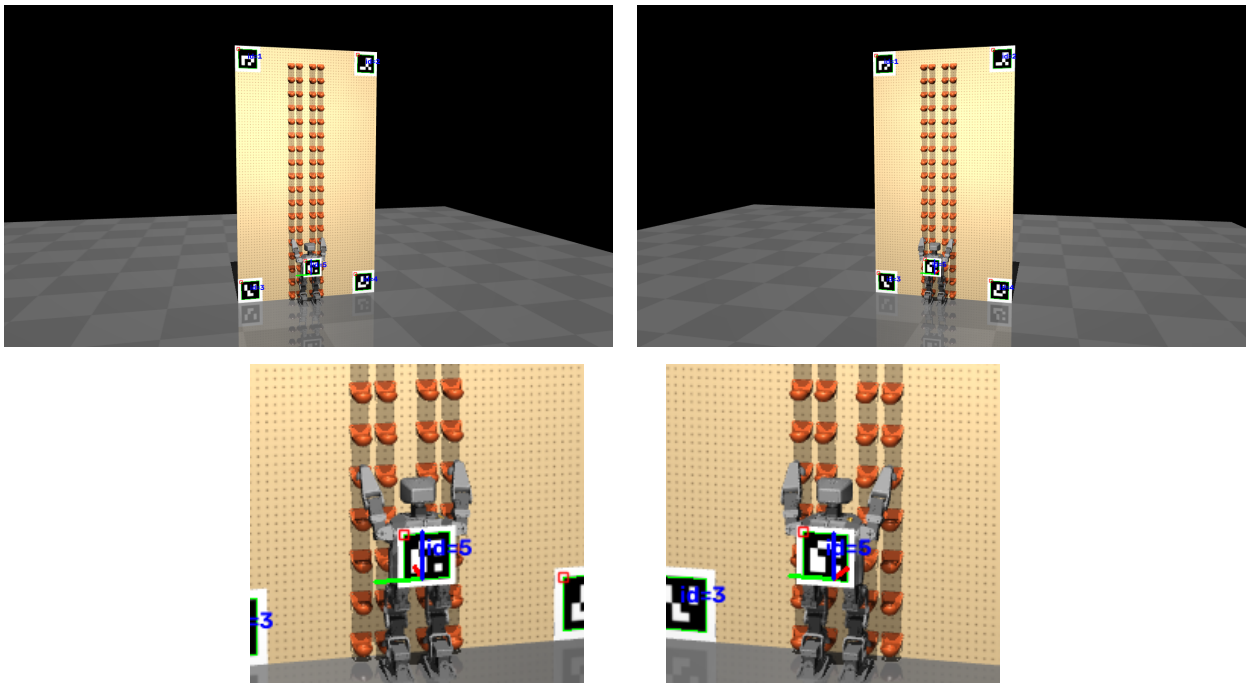


FIGURE 10 – Les images des deux caméras simulées, annotées (en haut), et un zoom sur le robot (en bas). Les cinq marqueurs sont détectés dans chaque image ; le repère du tronc calculé est dessiné sur le dos du robot.

10 Résultats

Toutes les mesures comparent la sortie du code à la vérité du simulateur. La simulation est déterministe : les mêmes commandes redonnent exactement les mêmes chiffres.

10.1 Pose des caméras (caméras par défaut, mur N1)

Caméra	Marqueurs vus	Reprojection	Erreur de position	Erreur d'angle
gauche	1, 2, 3, 4, 5	0,38 px	1,6 mm	0,033°
droite	1, 2, 3, 4, 5	0,38 px	2,3 mm	0,008°

10.2 Suivi du robot pendant qu'il se balance

Mur	Caméras	Position (mm)		Orientation (°)	
		moyenne	max	moyenne	max
N1	par défaut	1,6	3,2	1,53	3,44
N2	par défaut	1,2	2,5	1,02	2,32
N2	`--shuffle 1`	1,9	3,4	1,59	2,82
N2	`--shuffle 2`	2,1	3,3	1,63	3,58
N2	`--shuffle 3`	3,3	6,0	3,04	5,20
N2	`--shuffle 4`	1,7	3,2	1,37	2,89
N2	`--shuffle 5`	1,4	2,8	1,13	2,60

Pour tous les placements, les deux caméras voient les cinq marqueurs. Le moins bon cas (`--shuffle 3`) est celui où les deux caméras sont les plus rapprochées (73 cm d'écart) : comme prévu par la figure 7, la profondeur y est moins bien mesurée. Dans ce tirage, les caméras elles-mêmes sont localisées à 5–8 mm près (contre 2 mm par défaut), car elles sont plus loin et voient le mur plus de biais.

10.3 Le robot en haut du mur

Le robot a été accroché directement plus haut (sans grimper) pour vérifier que le marqueur reste bien vu quand les caméras le voient d'en dessous :

Mur	Hauteur du tronc	Erreur de position	Erreur d'orientation
N1	0,46 m	0,6 mm	0,41°
N1	1,14 m (haut du mur)	1,4 mm	0,82°
N3	1,12 m (haut du mur)	1,7 mm	1,73°

10.4 Une caméra contre deux (`live.py` sur des images du mur N3)

	x (mm)	hauteur (mm)	distance au mur (mm)
Vérité (simulateur)	385	172	116
Deux caméras	385	172	116
Une seule caméra	381	166	**95**

Avec une seule caméra, la distance au mur est fausse de 2 cm (c'est la direction de la profondeur pour la caméra), comme prévu par le calcul de la section 7.

11 Pour le vrai robot

1. **Brancher les deux caméras** en USB sur l'ordinateur de l'équipe. Les organisateurs les placent ; il ne faut pas les déplacer.
2. **Trouver leurs numéros** : lancer `live.py 0 1`, puis 1 2, 2 3… jusqu'à voir les deux images couleur. **Seule l'image couleur (RGB) est autorisée** par le règlement.

3. **Calibrer les intrinsèques** de chaque caméra (une fois) : soit lire les valeurs d'usine de la RealSense (`rs2_intrinsics`), soit faire une calibration OpenCV avec un damier. Les écrire dans un fichier `cams.json` :

```
{"cameras": [
  {"name": "cam_left",  "width": 1280, "height": 720,
   "fx": 910.2, "fy": 909.8, "cx": 641.3, "cy": 362.0, "dist": [0, 0, 0, 0, 0]},
  {"name": "cam_right", "width": 1280, "height": 720,
   "fx": 912.5, "fy": 911.9, "cx": 637.8, "cy": 359.4, "dist": [0, 0, 0, 0, 0]}]}
```

   (valeurs d'exemple). La clé `dist` n'est lue que par `live.py` : l'éditeur de caméras du simulateur ne la connaît pas, garde donc un fichier séparé pour lui.

4. **Imprimer le marqueur ID 5** (`marker_image()`) à 100 × 100 mm exactement, sur un support **rigide, plat et mat** (une feuille brillante fait des reflets), et le fixer sur le dos du robot. Vérifier à la règle que le code noir fait bien 80 mm.

5. **Mesurer sa position sur le tronc** et mettre à jour `P_TRUNK_MARKER` (et `R_TRUNK_MARKER` s'il n'est pas monté comme dans le simulateur).

6. **Vérifier la calibration** avec l'éditeur de caméras du simulateur : y charger une photo de la vraie caméra et les paramètres trouvés ; le mur dessiné par-dessus doit tomber exactement sur le vrai mur.

```
uv run climbing-sim-cameras
```

7. **À chaque essai** : robot en place, appuyer sur **c** pour recalibrer, vérifier que chaque caméra voit `[1, 2, 3, 4]` avec une reprojection inférieure à 1 pixel.

Ce qui n'a pas pu être testé

La lecture de vraies caméras USB n'a pas été testée : pas de matériel, et la webcam de l'ordinateur n'a pas été ouverte. Le premier essai sur place doit vérifier le numéro des caméras, la résolution (1280 × 720) et que l'image reçue est bien l'image couleur.

12 Limites et prochaines étapes

12.1 Limites connues

— **L'orientation du tronc** est moins précise (1 à 3°, jusqu'à 5° au pire) que sa position : les coins d'un marqueur de 80 mm, localisés au millimètre, laissent une incertitude de l'ordre de $1/40$ rad. Le robot a une centrale inertielle (IMU) qui donne son orientation (`obs["quat"]`, `obs["gravity"]`) beaucoup mieux : utiliser l'IMU pour l'orientation et le marqueur pour la position.

— **Occultation** : si un bras passe devant le marqueur dans une caméra, le code bascule sur l'autre caméra (précision réduite à quelques centimètres en profondeur) ; si les deux sont masquées, `update()` renvoie `None`. Le programme de grimpe doit le prévoir (garder la dernière pose connue, ou utiliser les angles des moteurs).

— **Simulateur idéal** : images sans flou, sans bruit, sans distorsion, éclairage parfait. Les vraies images seront moins nettes : la précision réelle sera moins bonne que ces chiffres.

— **Temps de calcul** : le rendu des images simulées coûte du temps ; sur le vrai matériel, la détection ArUco est rapide (quelques millisecondes par image).

— **Pénalité** : le mode avec marqueurs multiplie le temps par 1,25. Le mode sans marqueur demanderait de retrouver la pose des caméras et du robot sans aide (plus difficile).

12.2 Prochaines étapes

1. **Détecter les prises** dans les images. Les prises sont collées au mur ($z \approx 0$) : avec la pose des caméras, chaque pixel d'une prise correspond à un point unique du plan du mur. Il suffit ensuite de « coller » ce point sur la grille de trous (pas de 15 mm) pour avoir la position exacte de la prise. La couleur des prises (orange dans le simulateur) rend leur repérage facile.
2. **Écrire le programme de grimpe** : bouger un membre à la fois en gardant les trois autres accrochés, avec la cinématique inverse du simulateur, en s'appuyant sur la position du tronc donnée par le `Tracker` et sur la position des prises.
3. **Fusionner** position (caméras) et orientation (IMU) pour une pose du robot fiable à chaque pas de contrôle (20 ms).

A Aide-mémoire des commandes

```
cd ~/Desktop/Hackathon/simulator          # toujours depuis ce dossier

# demonstration (mur N1, N2 ou N3), cameras par defaut ou au hasard
uv run --with opencv-contrib-python python ../perception/demo.py N1
uv run --with opencv-contrib-python python ../perception/demo.py N2 --shuffle 3

# images annotees produites par la demo
open ../perception/out/cam_left.png ../perception/out/cam_right.png

# le jour J : cameras USB (gauche puis droite), avec ou sans calibration
uv run --with opencv-contrib-python python ../perception/live.py 0 1
uv run --with opencv-contrib-python python ../perception/live.py 0 1 --cameras cams.json

# sur des images, sans fenetre (ici les images brutes laissees par la demo)
uv run --with opencv-contrib-python python ../perception/live.py \
    ../perception/out/cam_left_raw.png ../perception/out/cam_right_raw.png --once

# verifier la calibration des cameras sur une photo (outil du simulateur)
uv run climbing-sim-cameras

# regenerer les images de ce document, puis le compiler
uv run --with opencv-contrib-python python ../perception/doc/make_figures.py
cd ../perception/doc && latexmk -xelatex perception_aruco.tex
```

B Référence des fonctions

Fonction (fichier)	Rôle
`detect(image)` `perception.py`	marqueurs d'une image RGB : `{ID: 4 coins en pixels}`
`wall_marker_corners(id)`	les 4 coins du code d'un marqueur du mur, en 3D (repère mur)
`square_corners(size)`	les 4 coins d'un marqueur carré dans son propre repère
`camera_pose(markers, K, dist)`	pose d'une caméra à partir des marqueurs 1–4 : `(rvec, tvec, rms)`
`marker_pose(views, size)`	pose d'un marqueur dans le repère du mur (triangulation, ou PnP avec une caméra)
`robot_pose(views, size, R, p)`	pose du tronc dans le repère du mur
`to_world(R, p)`	repère du mur → repère du simulateur
`annotate(image, tracker, nom, pose)`	image annotée (marqueurs + repère du tronc)
`Tracker`	le tout : `calibrate(images)` puis `update(images)`
`MarkedSim(...)` `robot_marker.py`	`ClimbingSim` avec le marqueur ID 5 sur le dos
`add_robot_marker(spec)`	ajoute la plaque du marqueur à une scène MuJoCo
`marker_image()`	image du marqueur ID 5 avec sa bordure (à imprimer)
`CODE`, `BORDER`, `PLATE`	tailles : 80, 10 et 100 mm
`R_TRUNK_MARKER`, `P_TRUNK_MARKER`	fixation du marqueur sur le tronc (à mesurer sur le vrai robot)

C Petit glossaire

Pose
position et orientation d'un objet (6 nombres : 3 pour la position, 3 pour la rotation).

Intrinsèques
paramètres propres à une caméra (focale, centre de l'image, distorsion) ; ne changent pas quand on la déplace.

Extrinsèques
pose de la caméra par rapport au monde (ici, au mur) ; changent dès qu'on la bouge.

PnP
(*Perspective-n-Point*) retrouver la pose d'une caméra à partir de points 3D connus et de leurs pixels.

Erreur de reprojection
écart, en pixels, entre les points détectés et ceux recalculés avec la pose trouvée ; mesure la qualité d'une calibration.

Triangulation
retrouver un point 3D comme l'intersection des rayons de deux caméras.

Kabsch
méthode qui trouve la rotation et la translation superposant au mieux deux nuages de points.

IPPE
méthode de PnP exacte pour des points dans un même plan (`SOLVEPNP_IPPE`), et sa variante pour un marqueur carré seul (`SOLVEPNP_IPPE_SQUARE`).

IMU
centrale inertielle du robot : gyroscope et accéléromètre, donne son orientation.